

Incrémenter / décrémenter les plats commandés

Projet **Délices de Douala** — TP Angular sur les `input()` et `output()` · 13 juillet 2026

Objectif : ajouter des boutons `+` et `-` sur chaque plat de la commande pour ajuster sa quantité en temps réel. L'enfant `LigneCommande` émet un événement (**OUTPUT**), le parent `Commandes` met à jour le signal.

Vue d'ensemble — qui fait quoi

1. `ligne-commande.ts` — déclare deux sorties `incrémenter` et `decrémenter` via `output()`.
2. `ligne-commande.html` — les boutons `+` / `-` émettent ces événements au clic.
3. `commandes.ts` — reçoit les événements et met à jour le signal `commandes`.
4. `commandes.html` — branche les sorties de l'enfant sur les méthodes du parent.

Étape 1 — `ligne-commande.ts`

On importe `output` et on déclare deux flux de sortie typés :

```
import { Component, input, output } from '@angular/core';
import { LigneCommande as LigneCommandeModel } from '../models/plat';

@Component({
  selector: 'app-ligne-commande',
  imports: [],
  templateUrl: './ligne-commande.html',
  styleUrls: ['./ligne-commande.css'],
})
export class LigneCommande {
  ligne = input.required<LigneCommandeModel>();

  incrémenter = output<LigneCommandeModel>();
  décrémenter = output<LigneCommandeModel>();
}
```

Étape 2 — `ligne-commande.html`

Les boutons appellent `.emit()` en transmettant la ligne courante :

```
<li class="ligne-commande">
  <span class="nom">{{ ligne().plat.nom }}</span>

  <span class="controle-quantite">
    <button (click)="decrémenter.emit(ligne())">-</button>
    <span class="quantite">x{{ ligne().quantite }}</span>
    <button (click)="incrémenter.emit(ligne())">+</button>
  </span>

  <span class="total">{{ ligne().plat.prix * ligne().quantite }} FCFA</span>
</li>
```

Étape 3 — commandes.ts

Le parent détient le signal `commandes`. On ajoute deux méthodes **dans la classe** :

```
incrementerLigne(ligne: LigneCommandeModel) {
  this.commandes.update((lignes) =>
    lignes.map((l) =>
      l.id === ligne.id ? { ...l, quantite: l.quantite + 1 } : l
    )
  );
}

decrementerLigne(ligne: LigneCommandeModel) {
  this.commandes.update((lignes) =>
    lignes
      .map((l) =>
        l.id === ligne.id ? { ...l, quantite: l.quantite - 1 } : l
      )
      .filter((l) => l.quantite > 0) // retire la ligne si quantité = 0
  );
}
```

Erreur fréquente : ces deux méthodes doivent être à l'intérieur des accolades de la classe `Commandes`, avant l'accolade fermante finale. Placées après, TypeScript signale une erreur.

Étape 4 — commandes.html

On branche les sorties de l'enfant sur les méthodes du parent avec la syntaxe `(evenement)="methode($event)"` :

```
<h2>Ma commande</h2>
@if (commandes().length > 0) {
  <ul class="commande">
    @for (ligne of commandes(); track ligne.id) {
      <app-ligne-commande
        [ligne]="ligne"
        (incrementer)="incrementerLigne($event)"
        (decrementer)="decrementerLigne($event)" />
    }
  </ul>
} @else {
  <p>Aucun plat sélectionné pour le moment.</p>
}
```

Résultat

1. Clic sur `+` → l'enfant émet `incrementer` → le parent augmente la quantité → l'affichage et le total se recalculent automatiquement (signal).
2. Clic sur `-` → même chaîne via `decrementer` → si la quantité tombe à 0, la ligne disparaît de la commande.